urdu-slm

Brief

A small Urdu language model trained from scratch: what it is, why it exists, what it proved, and what is wrong with it. The companion Deep Dive covers every module and function; this document is the argument, at altitude. A glossary of every technical term is at the end.

---

1 The claim, and whether it survives contact

Most "train a GPT from scratch" projects use English data and somebody else's tokenizer. This one picks a low-resource language and refuses both, which forces it to solve the problems the tutorials skip: script normalization, deduplication of a corpus that is mostly bot-generated stubs, language identification without a 126MB model, and a tokenizer that does not shred the language into individual bytes.

The headline claim is that GPT-2's tokenizer needs **5.23x more tokens** for the same Urdu text. That is not a rhetorical number; it is the project's whole justification, so it was checked rather than repeated.

Verified on this machine, 2026-07-16, to the digit

A real GPT-2 tokenizer was found in the local Hugging Face cache and `eval/tokenizer_compare.py` was run against it. Every figure in the README's table reproduced exactly: an 806,495-character held-out Urdu sample costs **197,758** tokens with this project's tokenizer (4.078 chars/token) against GPT-2's **1,034,831** (0.779 chars/token). Ratio: **5.23x**.

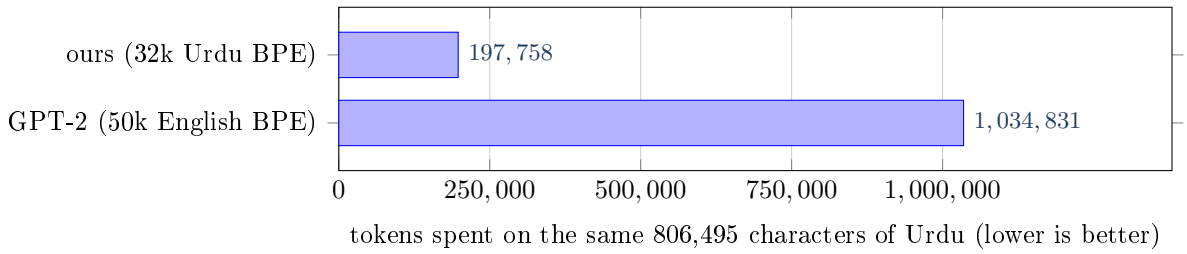

Figure 1: The measured compression gap, reproduced from the committed tokenizer and validation data.

1.1 Why the gap is that large

Two effects compound. UTF-8 gives every ASCII character one byte, but every Urdu letter lives in the Arabic block and costs **two**. And GPT-2's 50,257 merges were learned on English web text, so essentially none of them fire on Urdu byte sequences: the tokenizer falls back to its base alphabet and emits roughly one token per byte. At 0.779 chars/token it is spending more than one token *per Urdu character*.

Because every cost in the system is counted in tokens, this is not an aesthetic complaint. The GPU bill is per token. The context window is 1024 tokens, which holds about 4,176 Urdu characters with this tokenizer and about 798 with GPT-2's. And a model whose tokens are mostly raw bytes must spend its early capacity learning to reassemble characters before it can learn any Urdu at all.

The README's own command does not produce the README's number

The measurement is documented as `python -m eval.tokenizer_compare --gpt2 <path>`, but `--max-chars` defaults to 500,000. Run exactly as written it yields a 504,251-character sample and **5.14x**. Reproducing 5.23x needs `--max-chars 800000`. Both numbers are honest and the finding is identical either way; the defect is that the command printed beside the table is not the command that produced it. One flag fixes it, and it is worth fixing precisely because the number is genuine.

2 A note on the script

Why no Urdu appears in this document

Urdu uses the Perso-Arabic script: right to left, with context-dependent letter shapes. This PDF is built with `pdflatex`, which on this machine has no font for that script and no right-to-left engine; a multi-byte character inside a code listing is a hard build failure, not a rendering glitch. So Urdu text here is transliterated, described, or given as codepoints. The real script is in the repository's README and eval set, and renders fine in any editor. Nothing is hidden, it simply cannot be drawn here.

3 The system

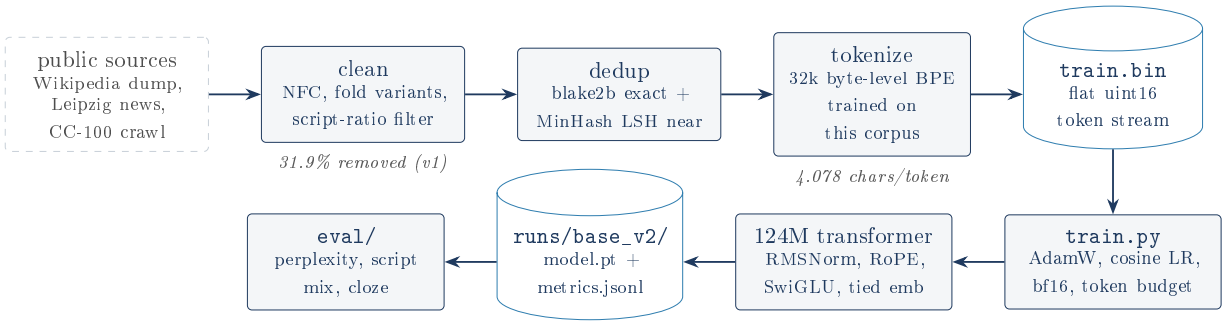

Figure 2: The whole project. Every stage streams to disk; nothing holds the corpus in memory. Training memory-maps the token stream.

Five components, about 2,800 lines of Python including tests:

- **The pipeline** streams raw text through normalization, filtering, deduplication and a deterministic train/val split, writing sharded JSONL. Each of its three stages writes a marker file so a rerun skips completed work.
- **The tokenizer** is a 32k byte-level BPE trained on the cleaned corpus, *train split only*, so no validation text shapes its merges.
- **The model** is a decoder-only transformer written out in one 186-line file with no modeling code imported from `transformers`. It is the Llama-family design, not a GPT-2 clone: RMSNorm, rotary position embeddings, SwiGLU, no biases, tied input/output embeddings, and `scaled_dot_product_attention` for Flash-kernel causal attention.
- **Training** budgets in tokens rather than steps, resumes exactly from a one-integer data cursor, runs bf16 on GPU and fp32 on CPU, and appends every metric to JSONL.
- **Evaluation** covers held-out perplexity, a script-mix diagnostic, a hand-written 95-item Urdu cloze harness, and the tokenizer comparison above.

3.1 Three architectural decisions worth defending

The vocabulary table dominates a small model, and the project says so

At a 32k vocabulary and 256 dimensions, the embedding table alone is 8.19M parameters; the six transformer blocks add only 4.82M. So the `tiny` preset is 13.01M, not the "10M" its config comment still claims, and 63% of it is vocabulary. The author tuned the blocks to a defensible 13M and documented the split rather than shrinking the vocabulary to hit a round number, which would have degraded the very tokenizer the project exists to demonstrate. All four presets' counts were re-measured here and match the README exactly: 13.01M, 35.66M, 123.69M, 336.38M.

SwiGLU at 8/3 is why `base` needs 14 layers, not 12

SwiGLU uses three weight matrices where a classic MLP uses two, so at the textbook 4x width it would cost 50% more parameters. The 8/3 ratio cancels that exactly ($3 \times 8/3 = 8 = 2 \times 4$). The consequence is that each layer is cheaper than a GPT-2 layer of the same width, so 12 layers do not reach 124M and the preset uses 14. Depth, not the memorized layer count, is what hits a target size. This was worked out, not copied.

The MinHash rewrite: a real 10x on a real bottleneck

Deduplication matters here more than usual because Urdu Wikipedia is largely bot-generated near-identical stubs, which is what produced 2.46M paragraphs from 1.73M articles. The library's per-shingle `MinHash.update()` loops in Python and measured ~250 docs/s, extrapolating to over two hours. The rewrite hashes all of a document's shingles with xxhash and computes the whole signature in two vectorized numpy operations, reaching ~2400 docs/s. The insight that makes it safe: MinHash signatures need only be *internally consistent*, never to match an external standard. That permits swapping datasketch's sha1 for xxhash and writing `hashvalues` directly, provided every document shares one scheme and one set of permutations, which reusing the seed object's own coefficients guarantees. The LSH banding on top is unchanged.

4 What it actually produced

Three real runs. Everything in this section was read out of the committed `metrics.jsonl` logs, not out of the README.

	tiny (CPU)	base v1	base v2
Parameters	13.01M	123.69M	123.69M
Steps	500	858	4,768
Tokens seen	4.10M	449.8M	2,499,805,184
Unique tokens	slice of v1	111.7M	862.4M
Tokens/parameter	n/a	0.9	20.2
Throughput	~1,600 tok/s	~99,470 tok/s	~105,000 tok/s
Train loss	9.74 to 5.42	9.59 to 3.51	9.63 to 2.74
Held-out perplexity	214.11 (re-measured)	38.02	**28.93**
Cost	free	~$1.90	~$13.06

Table 1: The three runs. Perplexity for the two base models is README-attributed (see the verification note below); everything else was read from the logs.

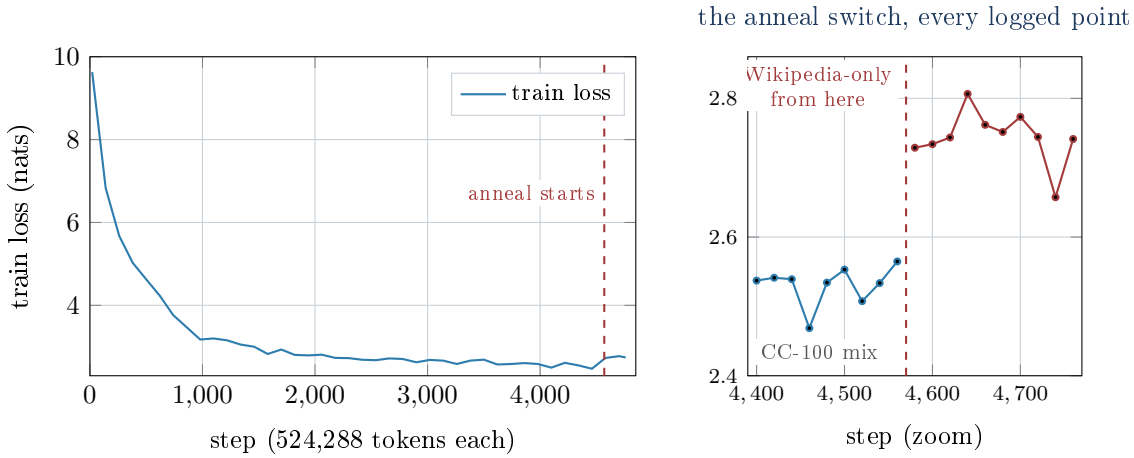

Figure 3: The v2 run, from the committed log. The step at the anneal boundary is real data. Its position corroborates the README's account: the main phase was ∼2.4B tokens, which at 524,288 tokens/step lands at step ∼4,578, and the log's jump is between 4,560 and 4,580.

4.1 The most important number is one the project refuses to claim

v2 gave the model 7.7x more unique tokens than v1 and moved the token/parameter ratio from 0.9 to 20.2, right at the Chinchilla-optimal ∼20. Perplexity fell from 38.02 to 28.93, a genuine ∼24% reduction. Cloze accuracy went from 34.9% to 39.5%.

And the README declines to call that a cloze improvement, because it is a 2-item difference on a 43-item set whose standard error is roughly ±7 points. It sits inside the noise band, and the README says so instead of presenting "+4.6 points" as a result.

The diagnosis, and why it is the best thinking in the project

The explanation given is correct and well-sourced: a 124M-parameter model has a real ceiling on how many discrete facts it can store, and that capacity scales with *parameter count*, not with training-text volume (Roberts et al., 2020). So 7.7x more text made the model more fluent, which is exactly what perplexity measures, without making it know more, which is what cloze measures. The corpus expansion did not fail; it did precisely what more text does. That diagnosis then drives the plan. The `medium` preset (336.38M, 2.7x the parameters) exists because parameters are the lever that moves fact storage. The Wikidata pipeline exists to supply dense declarative facts rather than diffuse prose. And the eval set was widened from 45 to 95 items *first*, cutting the noise band from ±7 to ±5.2, because the old set could not resolve the difference the next run is meant to produce.
Noticing that your headline metric is too noisy to support your next claim, and fixing the metric before running the experiment, is the most mature decision in this repository.

The v2 model writes essentially perfect Urdu script (99.75% of generated characters are Arabic script over 50 generations), which says the corpus filtering worked. What it does not have is reliable prompt-following: sample generations are fluent but drift off topic, and the README says so plainly rather than cherry-picking. Fixing that would need a larger model, fact-structured data, or an instruction-tuning pass, none of which this project attempts.

5 What is wrong with it

The Deep Dive lists thirty findings. These are the ones that would cost something to be surprised by. The pattern across almost all of them is the same, and it is an unusual one: **the code is in better shape than the prose around it**.

Upsampling repeats documents back-to-back inside one context window

`--upsample wikipedia:3` is implemented as `buf.extend(ids * reps)`, which is Python list repetition and lays the three copies down *contiguously* in the token stream. A typical kept paragraph is well under 100 tokens, so all three copies fit inside a single 1024-token training block. The model therefore repeatedly sees a context whose correct continuation is a verbatim copy of text ∼70 tokens earlier, which rewards copying rather than modeling. That is not what upsampling is meant to do; the intent is to make Wikipedia's *distribution* count for more. It shipped in the run that produced the headline model. Whether it measurably hurt cannot be established without a retrain. The fix is to repeat at the document-list level. This is the most substantive code finding.

The planned Wikidata data would contaminate the eval set the run exists to improve

`fetch_wikidata.py`'s docstring and `docs/DATA_LICENSES.md` both claim the fact triples use "different entities than the eval set's own items". Nothing enforces this, and it is false: the `capital` query selects the capital of *every country*, so it returns Pakistan/Islamabad, and cloze item 1 glosses as "The capital of Pakistan is Islamabad." Item 21 is covered the same way by the official-language query.

Two of 95 items, so the contamination is small, and Urdu Wikipedia already contains that fact in prose, so the eval was never clean in the strict sense. What is specifically wrong is a distinctness claim asserted twice and enforced nowhere, landing on the one metric the `medium` run is designed to move. Fixable in about thirty lines, before it matters.

The shipped config cannot reproduce the shipped model

`configs/base.yaml` says `max_tokens: 450000000` and its comments describe the superseded 111.7M-token v1 corpus. But `runs/base_v2` trained on 2,499,805,184 tokens, confirmed exactly against its own log ($4{,}768 \times 524{,}288$), plus an anneal phase on a different data directory. Both were done with CLI overrides that exist in `train.py`, but **the actual commands are recorded nowhere**, and `docs/GPU_RUN.md` still documents only `python train.py --config configs/base.yaml`, which would train a different model on a different budget. Someone following the documentation cannot reproduce the model the repository is about.

One number does not reproduce: the tiny run's perplexity

The README reports the tiny model's held-out perplexity as 248.8. The committed script, on the committed checkpoint and the committed validation set, gives **214.11** at the default coverage and **177.01** over the entire split. The script is fully deterministic, so this is not noise, and no batch budget produces 248.8.

The likely cause is visible in the timestamps: `runs/tiny/model.pt` was written at 20:21, but `data/val.bin` was regenerated at 21:46, over an hour later. The figure was measured against a validation set that no longer exists in the repository. Nothing was inflated (the real number is better than the claim), but in a project whose entire pitch is honest reporting, the one figure that does not reproduce is worth re-running.

`docs/GPU_RUN.md` is a pre-run plan whose predictions were falsified

It estimates throughput at "a conservative 30k tok/s" and states that "the only measured throughput in this repo is the CPU proof run". Two GPU runs have since measured 99,470 and ∼105,000 tok/s: the estimate was low by a factor of ∼3.3, and every cost projection in the document inherits the error. Its token-budget reasoning is v1-era throughout, and its

own advice ("add more data before spending GPU money") was taken and completed. Its post-run half is excellent, and that is why this is worth fixing rather than deleting: the RunPod SSH-key injection trap, the warning that installing the pinned `torch==2.12.1` silently replaces a pod's CUDA build with a CPU-only wheel, the `tmux` detach pattern, and the note that SSH cannot stop the billing meter so you must Terminate rather than Stop. That is hard-won operational knowledge. The document is two documents stapled together; splitting them would turn the weakest artifact into one of the strongest.

5.1 Shorter

- `generate` **has no KV cache**, so sampling is quadratic. Fine for a few 80-token diagnostics, disqualifying for serving. It is also why the grouped-query-attention branch has nothing to optimize: GQA shrinks a KV cache, and there is none. No preset enables GQA anyway, and its `repeat_interleave` implementation materializes the expanded tensors, so it saves no memory. The README's word "capable" is exactly right.
- **The DDP path has never been run.** Written carefully, including the gradient-sync suppression that cuts inter-GPU traffic 16x, but unproven.
- **The LSH index is entirely in RAM**, the one place the streaming discipline breaks. The README names this itself.
- **v1 never ran a single validation check**, because `eval_interval` (1000) exceeded the run's total steps (858).
- **All 95 cloze items have the answer at index 0.** Disclosed, and the harness scores log-probabilities without looking at position, so there is no leak. But it makes a whole class of bug undetectable: a position-bias regression would score 100% and look like a triumph.
- **Docstrings promise slightly more than the code delivers.** `train.py` claims resume restores RNG (the checkpoint contains no RNG state; near-harmless, since the data order comes from the restored cursor and dropout is 0). `cloze.py` claims "total log-probability" but computes a length-normalized mean, which is the better choice, undocumented.
- **Stale docs**: `evaldata/README.md` still says 45 items, `tiny.yaml` still says "~10M params", `base.yaml` still says "24GB GPU", and `data/corpus_stats.json` describes v1 only, with no committed stats for the v2 corpus the headline model trained on.
- `tqdm` **is declared in** `requirements.txt` **and never imported.**

What could not be verified here

The base models' downstream metrics (perplexity 38.02 and 28.93, script mix, cloze accuracy), the v2 corpus table, the GPU costs, and the Wikidata fetch counts are README-attributed. `data/train.bin` is gitignored (1.7GB, over GitHub's limit) and the v2 validation set is not committed, so those evaluations cannot be re-run without re-downloading ~1.3GB of sources and hours of pipeline. What *was* verified: the tokenizer's headline number to the digit, all 13 tests passing, all four presets' parameter counts, the v1 corpus funnel, all three runs' losses and throughputs, the v2 token count exactly, the anneal boundary's position, the v2 validation curve, and the eval set's shape.

6 The verdict

What this project demonstrates

The claim it is built on is real and reproduces exactly. The engineering underneath is sound and occasionally sharp: the MinHash vectorization is a genuine 10x with an articulable safety argument; the SwiGLU sizing was reasoned out rather than copied; training budgets in tokens so the scaling reasoning is expressible in the config; the causal-masking test tests the property by experiment and includes the negative assertion that stops it being a tautology; validation is never upsampled and the tokenizer never sees it.

The Urdu-specific decisions require actually knowing something about the orthography rather than about machine learning: keeping the zero-width non-joiner (it carries meaning) while stripping its invisible neighbours, and refusing to strip harakat because they matter in the minority of text that has them.

And the reporting is unusually honest, including where it is unflattering: the "10M" model is 13M and the README says why, the cloze gain is inside the noise band and the README refuses to claim it, and the eval set was widened to fix the noise *before* running the experiment that needs it.

The weaknesses are almost entirely documentation drifting behind code: a GPU plan never updated after the run falsified it, configs that no longer describe the model that shipped, docstrings overreaching slightly, and a perplexity measured against a validation set that was later regenerated. Two real code defects (contiguous upsampling, and the unenforced Wikidata distinctness claim) are worth fixing before the next run. That is the opposite of the usual failure mode, and a much easier problem to have.

7 Glossary

Annealing
Finishing training on a small, high-quality data slice after a large mixed corpus, so the final steps bias the model toward the better distribution. v2 ended with a ∼100M-token Wikipedia-only phase; the switch is visible as a loss step in the log.

Attention
The mechanism letting each position look at others. Each position emits a query, a key and a value; queries are compared against all keys, and the resulting weights average the values.

bfloat16 / AMP
A 16-bit float keeping float32's exponent range but fewer mantissa bits. It cannot overflow where fp32 would not, so it needs no loss scaling. Automatic Mixed Precision runs most operations in it while keeping master weights in fp32.

BPE (Byte-Pair Encoding)
Tokenizer training: start from single bytes and repeatedly merge the most frequent adjacent pair into a new vocabulary entry until reaching the target size (32,000 here). *Byte-level* BPE seeds the vocabulary with all 256 bytes, so no input can ever be out-of-vocabulary.

Causal masking
Restricting each position to attend only to itself and earlier positions. Without it the model could read the answer it is asked to predict.

Chinchilla-optimal
The rule of thumb that compute-optimal training uses ∼20 tokens per parameter. v1 was at 0.9 (badly data-limited); v2 reached 20.2.

Cloze
A fill-in-the-blank test. The model scores each candidate completion rather than generating; the best-scoring one is its answer.

Context window
The maximum tokens the model attends over at once: 1024 here. Attention cost grows with its square.

Cross-entropy loss
The training objective, in nats: $-\ln p$ for the true next token. Lower means less surprised.

DDP
DistributedDataParallel: each GPU holds a full model copy and a different batch slice; gradients are averaged after each backward pass.

Deduplication, exact and near
Exact uses a blake2b hash. Near uses MinHash+LSH to catch documents that differ slightly, which exact hashing misses entirely.

Embedding
The lookup table turning a token id into a vector. At 32k vocabulary it dominates small models: 63% of `tiny`.

FlashAttention
An exact attention implementation that never writes the full position-by-position matrix to GPU memory, tiling it through fast on-chip memory instead.

Gradient accumulation
Running several micro-batches that each add into the same gradient buffers before one optimizer step, giving a large effective batch without the VRAM.

GQA (grouped-query attention)
Sharing keys and values across groups of heads to shrink the generation-time KV cache. Implemented here but enabled by no preset.

Jaccard similarity
Intersection over union of two sets. Here, of two documents' shingle sets. The near-dedup threshold is 0.8.

JSONL
One complete JSON object per line; readable and appendable one record at a time.

KV cache
Storing past keys and values during generation so each new token costs $O(1)$ rather than a full re-run. This project has none, so sampling is quadratic.

LSH
Locality-Sensitive Hashing: bucket signatures so only plausibly-similar documents are ever compared, avoiding a quadratic scan.

Memory-mapping
Presenting a file as if it were an in-memory array, loading pages on demand. How a 1.7GB corpus trains without a 1.7GB allocation.

MinHash
A fixed-size signature (64 numbers here) whose agreement rate estimates Jaccard similarity, computed without comparing the sets.

Nats
The unit of loss when using natural logarithms. Perplexity is e^{nats}.

Perplexity
e^{loss}. Roughly "how many equally-likely options did the model feel it was choosing between". v2 scores 28.93.

Pre-norm
Normalizing a sublayer's input rather than its output, leaving the residual path clean. Markedly more stable than post-norm.

Residual connection
`x = x + sublayer(norm(x))`: the sublayer computes a correction rather than a replacement, giving gradients a clean path to early layers.

RMSNorm
Normalization dividing by the root mean square with one learned scale, dropping LayerNorm's mean-subtraction and shift. Computed in fp32 here for stability.

RoPE
Rotary position embeddings: rotate queries and keys (never values) by an angle proportional to position, so attention scores depend only on relative distance. Zero parameters.

Shingle
A short overlapping substring (5 characters here). Documents sharing most shingles are near-duplicates.

SwiGLU
A gated MLP: the input is projected twice, one path passes through SiLU and multiplies the other. At an 8/3 ratio it matches a 4x GELU block's parameter count.

Token
The unit a model reads and writes: a frequently occurring chunk of characters, not a letter and usually not a word.

Weight tying
Using one matrix for both the input embedding and the output projection. Saves 24.6M parameters in `base` and usually improves quality.

ZWNJ
Zero-width non-joiner (U+200C): invisible, but carries real orthographic meaning in Urdu. Deliberately kept where its invisible neighbours are stripped.